

# Multi-Agent Systems

Architectures · Supervisor Pattern · Building a GitHub Agent

## 1. Topic Overview

Session 8 was the point where everything from the previous sessions came together. Until now the cohort had built single agents and stateful LangGraph workflows. This session took that same set of building blocks, state, nodes, edges, tools and the ReAct loop, and multiplied them into a **multi-agent system**: several specialised agents working under a coordinator to finish a task that one agent would struggle with.

The session had two halves. The first half was architecture: what a multi-agent system actually is, why you would reach for one, when it is an overkill, and the three patterns you will use most often (sequential, coordinator or supervisor, and hierarchical). The second half was a long hands-on build. Everyone created a real GitHub App, wired it to a LangChain GitHub toolkit, and shipped a supervisor-pattern agent that wrote documentation into a live repository, created a branch and opened a pull request. The mentor was clear about the framing throughout: a multi-agent system is not a badge of sophistication, it is a cost and accuracy trade-off you make deliberately.

| Topic                        | What was covered                                                                                                        |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Recap of the building blocks | State, nodes, edges, human in the loop, subgraphs, and the ReAct loop that runs inside every agent.                     |
| What a multi-agent system is | Multiplying the single agent architecture into specialised copies, each isolating its own context.                      |
| Why multi-agent              | Scaling test time compute, specialisation, parallelisation, and solving problems a single agent cannot.                 |
| When NOT to use it           | A decision table across task complexity, speed and latency, cost, and accuracy.                                         |
| Three core architectures     | Sequential, coordinator (supervisor or orchestrator), and hierarchical, with a full comparison.                         |
| Hands-on build               | A GitHub multi-agent system: GitHub App setup, LangChain GitHub toolkit, sub-agents wrapped as tools, and a supervisor. |
| Prompt design                | The CRAFT structure for writing sub-agent system prompts.                                                               |

| Topic              | What was covered                                                                       |
|--------------------|----------------------------------------------------------------------------------------|
| Production nuggets | Reliability, state management, human in the loop done right, security, and evaluation. |

### Industry Context

The tools the cohort already uses are multi-agent systems. Claude Code spins up sub-agents on its own when it decides a task should be split. OpenClaw does the same. CrewAI, which the mentor put in the chat during the session, follows a hierarchical structure by default. The project built in this session was inspired by Code Rabbit, an agentic platform that plugs into your GitHub repository and reviews pull requests automatically, and which existed well before Claude Code.

## 2. Key Concepts

### 2.1 From one agent to many

An AI agent is a program that takes input, thinks about it, and acts to complete a task using tools, memory and knowledge. Internally it runs the ReAct loop: it reasons using the LLM, and it acts using the tools it has access to. That is the whole shape of a single agent, an LLM with tools, knowledge and memory wired into it.

A multi-agent system simply multiplies that architecture. You create copies of the agent, and each copy specialises in one task. The immediate benefit is **context isolation**. When you break one agent into sub-agents, each sub-agent only carries the context it needs, which reduces the cognitive overload on the model. A smaller model does not have to think as hard to do a smaller task. It is divide and conquer applied to LLMs, and it lets you solve complex problems a single agent would fail at.

### Key Insight

Remember the phrase "isolating context". It is the reason multi-agent systems work at all, and it comes back later in the session as the reason you return only the last message from a sub-agent instead of its whole conversation.

### 2.2 Why multi-agent: scaling test time compute

The first and most important reason for multi-agent systems is to scale **test time compute**. The term became popular around the DeepSeek launch. The idea is that you do not spend all your money during training. During training you teach the model generic tasks. Then at test time, during inferencing itself, you apply methods that let the model retrieve context in a way that helps it complete difficult, long running tasks.

Multi-agent systems are one way to scale test time compute. You take a complex problem, use multiple instances of the LLM, and generate far more reasoning and intermediate data at inference time, in parallel, than a single call ever could.

| Benefit                          | What it means in practice                                                                                                                                               |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Scaling test time compute</b> | Spend compute at inference across multiple LLM instances instead of relying on one pass.                                                                                |
| <b>Specialisation</b>            | Each agent is tuned to one job, for example a research agent and a summariser agent, or a booking agent broken into itinerary, price search, booking and update agents. |
| <b>Parallelisation</b>           | Widen the search space. A research agent can run one sub-agent on YFinance, one on Google, one on Money Control, then consolidate the results for the supervisor.       |
| <b>Context isolation</b>         | Each agent carries only what it needs, which reduces the load on the model and the chance of the context being poisoned.                                                |

One point the mentor corrected on the spot: a learner suggested that multi-agent systems save time. They usually do not. Every hand-off from a coordinator to a sub-agent costs tokens and takes time to delegate, and the back and forth between them consumes both. Multi-agent systems buy you accuracy and the ability to attempt harder problems, not speed.

## 2.3 When to use a multi-agent system, and when not to

This was the slide the mentor asked everyone to screenshot, because it is the one you use to make architecture decisions. Four criteria, and each one points to a single agent or a multi-agent system.

| Criteria                 | Choose a single agent when                                                       | Choose a multi-agent system when                                                     |
|--------------------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <b>Task complexity</b>   | The task is straightforward. A multi-agent system here is an overkill.           | The task is complex and can be decomposed into multiple smaller tasks.               |
| <b>Speed and latency</b> | You want fast execution. One LLM, a simple set of tools, no delegation overhead. | Speed is not the priority, and a single agent would hit its context limit.           |
| <b>Cost</b>              | Cost matters more to you than squeezing out extra performance.                   | Accuracy matters more than cost.                                                     |
| <b>Starting point</b>    | Always. Start here, every time.                                                  | Only after you have identified the concrete limitations of the single agent version. |

### Key Insight

"Always, always start with a single agent system. Identify the limitations of it. And then decide whether or not you need to pivot to a multi-agent system." Do not start with the most complex architecture and try to simplify it later. Scale up, do not scale down.

The same question came up in the chat in a very concrete form. Sibasis is automating an eight step GCP load process involving Excel comparison and database queries. His manager wants a multi-agent system, and he thinks a single agent will do. The answer from the team was the general rule applied: if the order is fixed and sequential, a sequential workflow is better. Only if you need dynamic routing, or you are hitting context limits, do you move to multi-agent. And you prove it by comparison, experimentation and evaluation, not by argument.

## 3. The Three Multi-Agent Architectures

Before going into the patterns, one caveat the mentor gave: these are the basic architectures that early AI engineers converged on. You can always design a custom architecture. There is also no problem where one of these cannot be used. The only real question is which one is **most suitable**. The mentor deliberately took one example, a research agent that drafts, reviews and finalises, and re-drew it in each pattern to show the same use case can be built three ways, and that two of those ways are usually wrong.

### 3.1 Sequential pattern

The simplest one, and the one the cohort had effectively already built in the previous session. Agent A produces output, hands it to Agent B, Agent B hands to Agent C, and execution happens one by one. When everything has run, the last sub-agent returns the response to the main agent, which replies to the user.

The research example fits it cleanly: a research agent generates a draft, a review agent reviews it, a finaliser agent finalises it. Because there is no delegation loop, this is the pattern that consumes the least resources and the fewest tokens of any multi-agent design.

The class suggested several use cases in the chat, and the mentor sorted them live. Code review, email generation, invoice processing and data extraction, and trip or ticket booking all fit. Document reading, stock trading and production error monitoring were rejected as poor fits for a purely sequential design.

One clarification worth keeping: there is no scope for parallelisation **inside** a sequential pipeline, because each agent is waiting on the previous agent's output to begin. You can run several sequential pipelines in parallel, but not the agents within one.

### 3.2 Coordinator pattern (supervisor or orchestrator)

Here you have a boss agent. Sub-agent A and sub-agent B both report to the coordinator, and the coordinator dynamically decides which one to delegate the job to. Think of a small team: a team lead assigns the task, assesses the output, modifies it if needed, and passes it on.

Two properties matter. First, **the sub-agents never talk to each other**. All communication goes through the coordinator. Second, each of those sub-agent blocks is itself a ReAct agent running its own loop. Sub-agents can be made to run in parallel, but usually they do not, because one agent's output often depends on another's. Run them in parallel only when there is genuinely no dependency.

#### Key Insight

The coordinator pattern is also called the orchestrator pattern, because the coordinator orchestrates the workflow by choosing which sub-agent to delegate to. This is the pattern the cohort built by hand in the hands-on section.

### 3.3 Hierarchical pattern

Scale the coordinator across multiple levels and you get a hierarchy. A coordinator has sub-agents, and each sub-agent can itself be a coordinator of further sub-agents, exactly like a real organisation with multiple layers of management. OpenClaw was the example the class landed on, and Claude workspaces can do this too. It fits anywhere you are building a digital twin of a company, for example a marketing agent or a content creation agent that fans out into specialists.

This is the most expensive architecture to build, because of the volume of conversation flowing between agents. It is also the least predictable, and the mentor explained why with an analogy the whole class recognised: **Chinese whispers**. One person whispers a word around a circle, and by the time it returns the meaning has changed completely. That is what happens in a hierarchical agentic system. If a small amount of context poisoning happens mid execution, it can change the whole direction of the run, and it can even change the goal.

Sandip pushed back in the chat: how does the Chinese whispers analogy fit, when the sub-agents are not talking to each other? The answer is that the corruption travels through the manager. If one agent gets context poisoned and returns an irrelevant response to the manager agent, and you have not engineered it well, that message lives on in the manager's history and trickles down into every other sub-agent it delegates to afterwards.

### 3.4 Comparing the three patterns

This is the second slide the mentor asked the class to screenshot. It is reproduced here in full, and also exists as a standalone reference card alongside these notes.

| Dimension        | Sequential                                                                          | Supervisor                                                    | Hierarchical                                                                      |
|------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Use when         | The steps are always in the same order and nothing needs to be decided at run time. | The right next step depends on the task and may loop or skip. | You have multiple distinct domains, for example a digital company with sub-teams. |
| Number of agents | Few, in a fixed chain.                                                              |                                                               |                                                                                   |

| Dimension                 | Sequential                    | Supervisor                                                                | Hierarchical                                                           |
|---------------------------|-------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------|
|                           |                               | Few and fixed, typically 3 to 6 workers plus the supervisor.              | Many, across several layers.                                           |
| <b>Predictability</b>     | Highest. Least probabilistic. | Lower than sequential, but still workable.                                | Lowest, because of context poisoning travelling through the hierarchy. |
| <b>Cost and latency</b>   | Most budget friendly.         | More expensive, acceptable when you need accuracy on a complex task.      | Most expensive, and runs for the longest.                              |
| <b>Debuggability</b>      | Easiest to debug.             | Harder. You have to track the conversations to find the point of failure. | Hardest. Points of failure grow exponentially.                         |
| <b>Engineering effort</b> | Low.                          | Moderate.                                                                 | High. Needs thorough context engineering to be safe.                   |

CrewAI came up here as the well known example of a framework that defaults to a hierarchical structure. It became popular because people could seemingly spin up multiple sub-teams with it. The mentor's advice was to be extra careful when reaching for hierarchical, precisely because of the predictability and cost profile above.

## 4. Hands-On: Building a GitHub Multi-Agent System

### 4.1 The problem statement

The class built a simplified version of **Code Rabbit**, an agentic platform that integrates with your GitHub repository and can make code changes, review pull requests and do all of that automatically. The agent needed to write documentation into a real repository, create a branch, and open a pull request, all without a human touching the repo.

The integration used was the **GitHub Toolkit from LangChain community**. The mentor was upfront that langchain-community has been archived and is being sunset, and that using it here was deliberate: in a later session, after the MCP lecture, the same project will be migrated to the current LangChain specification. Being able to do migrations is an essential skill, because the ecosystem is still moving fast.

## 4.2 Setting up the GitHub App

Before any code, everyone created a GitHub App. The App is how the agent authenticates and acts on the repository. The chain is: **your agent, to the GitHub App, to the installation, to the specific repository**. The agent you build is effectively the backend of that GitHub App.

| Step | What to do                                                                                                                                                                                       |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Create a GitHub account or log in, then create a dummy public repository with a few files. This was the to-do given before the break.                                                            |
| 2    | Clone the reference project: <code>github.com/tnahddisttud/github-agent</code> . Clone it anywhere you keep your projects. This is not the repo the agent will act on.                           |
| 3    | Go to your profile, Settings, then Developer settings, then GitHub Apps, and click New GitHub App. Give it a unique name, the mentor's comparison was that app names are like Instagram handles. |
| 4    | Set the homepage URL to your GitHub profile URL, and uncheck the active webhook. You are not creating webhooks.                                                                                  |
| 5    | Under Repository permissions set Contents to read and write, Issues to read and write, and Pull requests to read and write. Metadata is read only and mandatory, you cannot change it.           |
| 6    | Create the app, then copy the App ID from the resulting screen.                                                                                                                                  |
| 7    | On the General tab, scroll to Private keys and click Generate a private key. A .pem file downloads to your downloads folder.                                                                     |
| 8    | Go to Install App, install it on your account, choose Only select repositories, and pick the dummy repository you created in step 1.                                                             |

Two things caused the most confusion in the room and are worth stating plainly. The "Danger zone" box that appears after installation is not an error, it just means those options can uninstall or suspend the app. And the repository you select at install time is the repository the agent will write into, which is the one you created, not the reference repo you cloned.

### Key Insight

Why read and write on Contents? Because contents is everything in your repository, and the agent needs to be able to change files in order to contribute. Why read and write on Issues and Pull requests? Because we want the agent to read, triage, update and solve issues, and to raise pull requests. Every permission is granted for a specific capability, nothing more.

## 4.3 Project and environment setup

The cloned project is a **uv** project, so it ships with a `pyproject.toml`, a `uv.lock`, a `main.py`, a Python version file and a `.gitignore`. `uv` is a package manager, in the same family as `npm`, `Maven` or `Gradle`. If `uv` is not installed,

install it from the official uv installation docs for your operating system rather than through pip, so the uv environment is not tied to any particular Python or pip install.

| Command or file               | Purpose                                                                                                                                         |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>uv sync</b>                | Installs all dependencies and creates the .venv. Run it from the folder containing pyproject.toml, otherwise you get "No pyproject.toml found". |
| <b>.venv\Scripts\activate</b> | Activates the virtual environment on Windows. On macOS and Linux use <code>source .venv/bin/activate</code> .                                   |
| <b>Ctrl and tilde</b>         | Opens the integrated terminal in VS Code or Antigravity.                                                                                        |
| <b>.env</b>                   | Create your own from sample.env. Holds GITHUB_APP_ID, GITHUB_APP_PRIVATE_KEY, GITHUB_REPOSITORY and GROQ_API_KEY.                               |
| <b>resources/</b>             | A folder you create to hold the .pem private key. Not strictly required, but it keeps the project tidy.                                         |

The packages installed are langchain, langchain-community, langchain-core, langchain-groq, langgraph, pygithub and tiktoken. PyGithub is there because the LangChain community GitHub integration uses it under the hood.

For the environment variables: GITHUB\_APP\_ID is the App ID you copied, GITHUB\_APP\_PRIVATE\_KEY is the **path** to your .pem file and never its contents, GITHUB\_REPOSITORY is the repository you selected at install time in `username/repo-name` form and not a full https URL, and GROQ\_API\_KEY is your own Groq key.

#### Key Insight

A .env file holds private, confidential credentials. That is exactly why the project ships a sample.env with placeholders instead. Never share your .env, never paste API keys into chat, and never commit either the .env or the .pem file to a repository. Add both to .gitignore.

## 4.4 Warm-up: the release notes workflow

Before the agent, the mentor walked through a plain LangGraph workflow in the project, a release notes generator. It is a good contrast, because release note generation does not need agents at all as long as the project is not very complicated. It just checks which commits landed after the previous release and writes notes for them.

The build order is the one the cohort already knew, and the mentor asked the class to supply each step:

- **Define the state first.** Here it holds release info, a list of PRs and commits, and a release notes string. That is what the workflow populates.

- **Instantiate the LLM** (ChatGrok) and the `GitHubAPIWrapper`, then pass the wrapper into `GitHubToolkit.from_github_api_wrapper()` to get the tools. All of this comes straight from the documentation.
- **Build a tool map**, a simple dictionary with the tool name as key and the tool itself as value.
- **Write plain Python functions** for each step, for example collecting PRs and commits with the list pull request tool, and fetching previous releases.
- **Convert functions to nodes** with `graph.add_node(name, function)`. The function is the executor of that node.
- **Connect them with edges**. Adding nodes alone connects nothing. Start from the START node, chain through, and finish at END. With no conditional edges, this is a sequential graph. Add an edge back from a later node to an earlier one and you have created a cycle.
- **Compile** with `graph.compile()` and invoke.

The mentor then used this to define a term precisely. If you take one of those node functions and, instead of calling a tool directly, have it call an agent that invokes a bunch of tools, you have not built a multi-agent system. If only one node is agentic, it is an **agentic workflow**: the main program is a workflow, but some nodes have agency.

## 4.5 utils.py: renaming tools and least privilege

The GitHub toolkit gives you tools with human friendly names like "Get Issues" and "Comment on Issue". Those names are not friendly to an LLM. Agents are trained to treat tools as functions, so a tool name cannot contain spaces. The first utility in `utils.py` is therefore `rename_tool`, which walks a name mapping and renames every tool from its human readable form to an AI native, function-like form.

The second set of utilities fetches only the relevant tools for a given task. There is a helper for issue triage that returns the issue tools, one for code review that returns the read and search tools, and so on. The mentor asked whether a code review agent needs the write file tool. It does not. It needs the read and search tools only.

### Key Insight

This is the **principle of least privilege**, and the point the mentor wanted to land is that it should not apply only to your users. It should apply to your agents too. Only hand an agent the list of tools it actually requires. It keeps your agent code clean and it limits the blast radius of a misbehaving agent.

There is a second, quieter lesson embedded here about code hygiene. The mentor's rule of thumb, delivered as a joke that stuck: write your repository as if the next person to develop on that code is a psychopathic serial killer who will come for you if you left a mess. Keep it clean and tidy.

## 4.6 graph.py: sub-agents, agents as tools, and the supervisor

With the tools renamed and grouped, the agent file follows the same opening steps: instantiate the LLM, create the `GitHubAPIWrapper`, get the toolkit, pull the list of tools and run every one of them through `rename_tool`.

Then the sub-agents are created, each with LangChain's `create_agent` function. Four of them were built in the session:

| Sub-agent                  | Responsibility and tools                                                                                                |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Documentation agent</b> | Writes documentation into the repository. Later in the session the create pull request tool was added to its tool list. |
| <b>Release notes agent</b> | Gets only the release notes tools from the helper function.                                                             |
| <b>Issue agent</b>         | Gets only the issue tools, for reading, triaging and updating issues.                                                   |
| <b>Code review agent</b>   | Gets only the read and search tools, deliberately no write tool.                                                        |

Each is defined with a leading underscore in its variable name. That is a convention for marking a sub-agent, not a requirement. Removing it changes nothing functionally, but it tells the next developer what they are looking at.

Each sub-agent takes three things: the model, the tools returned by its helper function, a name, and a system prompt describing its specialised task.

The step that turns this into a supervisor system is what happens next. Every sub-agent is **wrapped in a plain Python function that is annotated with the `@tool` decorator**. Inside that function the sub-agent is invoked, and its response is converted into a tool message and returned as a `Command`. In LangChain and LangGraph, `Command` is the mechanism for agent to agent communication, and more broadly for injecting a message into an LLM pipeline from an agent, a tool or a middleware. The cohort had already met it in the human in the loop session.

Crucially, only the **last message** from the sub-agent is wrapped and returned, never the whole result. The mentor asked the class what would happen if the full result were returned, and the chat answered correctly: the context would blow up. The framing that made it click: your manager does not need to know every step you followed, only whether the task was completed well. This is context management, and it is the practical application of the "isolating context" idea from the start of the session.

Finally the supervisor. It is another `create_agent` call, with the LLM, a supervisor system prompt that delegates tasks, and a tools list where **every tool is one of those wrapped sub-agents**:

### Key Insight

The relationship between supervisor and sub-agents is not established by naming something "supervisor". It is established by passing the four wrapped agent functions into the supervisor's tools list. In the supervisor pattern, **each agent acts like a tool**. That single line in the code is the entire architecture.

This is a supervisor pattern, not a hierarchical one, because none of the four sub-agents has sub-agents of its own. It is worth noting what was **not** programmed: the sequence. The supervisor decides on the fly which agent to call and in what order, based on the task it was given. That is what makes it agentic rather than a workflow.

## 4.7 Running it, and what actually happened

The agent was invoked with a task that required two steps: create documentation, and raise a pull request for it. The run command, after the first attempt failed with a relative import error, was:

```
uv run python -m src.agents.graph
```

The model used was `openai/gpt-oss-120b` through Groq. The mentor was candid that this is a fairly primitive model for a task like this, so a first or second run may fail and it is worth retrying. Results in the room were genuinely mixed, which turned out to be the most useful teaching moment of the hands-on.

The mentor's own first run created a new branch and added a README, but did not open a pull request. Several learners got a pull request on the first try. The reason, which the mentor worked out live by reading the agent's trace, was that the documentation agent did not have the create pull request tool in its tool list. Adding `create_pull_request` to the documentation tools in `utils.py` and re-running produced the pull request. The learners who got a PR without that change got it because their model happened to work out that the code review agent had access to the PR tool and routed there instead. Same code, different outcome, because the system is stochastic.

### Key Insight

How did the mentor find the bug? By reading the trace of what the agent had done, not by guessing. The trace showed the agent never had the tool it needed to perform the action. Observability is not an optional extra in agentic systems, it is how you debug them at all. Session 9 covers this properly.

The agent never received an instruction to create a branch. It was told to add documentation and create a PR, and it created the branch on its own. The mentor pointed at this later as an accidental but real example of human in the loop: the pull request itself is the human checkpoint.

## 4.8 Debugging with the IDE assistant

A large number of learners hit errors during the hands-on. The mentor's recommended workflow, which several learners called the most useful thing they learned in the session, was to stop guessing and hand the error to the IDE assistant. Select the error text in the terminal, click the chat icon in Antigravity or your IDE, attach or

paste in the setup documentation, and ask it to fix the error based on that documentation. The pre-read PDF also carries a common errors and remediation section.

## 5. Writing Sub-Agent System Prompts: CRAFT

Karthik Prabhu asked how the system prompts for each sub-agent were designed. The mentor's answer was that a system prompt has an order to it. First give the agent a **role**: you are a code review agent, you are an issue management agent, you are a release notes agent, you are a documentation agent. Then give it the task it is supposed to do. Then specify the actions it can perform, which in practice means the tools it has access to.

The named method for this is **CRAFT**. Sunil posted the full expansion in the chat, and the mentor endorsed it.

| Letter                   | What it sets                                                                                        |
|--------------------------|-----------------------------------------------------------------------------------------------------|
| <b>C, Context</b>        | The background, target audience or operational environment, so the model understands the situation. |
| <b>R, Role</b>           | The exact expert persona or identity the agent should adopt.                                        |
| <b>A, Action</b>         | The precise tasks, goals or core behaviours the agent must perform.                                 |
| <b>F, Format</b>         | The structure, length limits or layout of the output.                                               |
| <b>T, Tone or Tweaks</b> | The voice, plus explicit rules and counter-examples, for example "no corporate jargon".             |

One small practical note from the same part of the walkthrough: if you skip the docstring on a function you are turning into a tool, you can pass a description into the `@tool` decorator instead. Writing a proper docstring is the better habit.

## 6. Production Nuggets: Building Multi-Agent Systems That Hold Up

The final teaching block was a set of hard-won practices, drawn from the mentor's own experience and from conversations with AI engineers at big tech companies. Most of these get a full session of their own later in the curriculum, so treat this as the map.

### 6.1 Reliability

One bad output can break the whole system. If the coordinator picks one wrong route, the entire downstream output changes. To contain that:

- **Enforce structured output.** This is where states earn their keep, and LangGraph gives you them. Pydantic or JSON schemas plus validation before results pass from one agent to the next.

- **Add validation gates between agents**, implemented as middlewares, which is a topic in a coming lecture.
- **Cap the number of retries** an agent can make, and add backoff, timeouts, and a defined stage past which the agent does not continue.

Infinite loops are a real risk, not a theoretical one. The mentor described his own agent running for around sixteen minutes on Groq because of a middleware bug. He had used a summariser to reduce context propagation, but it was trimming rather than summarising, cutting out the meat of each tool's output. The agent concluded the tool execution had failed and kept retrying. You detect this class of problem with max iterations and timeouts, and you find the cause by reading the trace.

## 6.2 State management

- **Avoid race conditions.** If several agents read and write a shared state in parallel, add checkpointing or memory so that whatever is happening while one agent holds the state is saved.
- **Treat states as immutable** and build the pipeline around that assumption.
- **Design each node to be idempotent.** The mentor's analogy was `CREATE TABLE IF NOT EXISTS`. Running it again does not change the result. Do not allow individual nodes to update state without proper write validation.

## 6.3 Human in the loop, done right

Human in the loop comes in various flavours, and in the GitHub agent it arrived through the pull request. The agent was never told to create a branch, but the constraint "do not change anything directly, raise a PR" is itself a human checkpoint. Just as you block humans from pushing straight to main, block your agents too.

The failure mode to design against is a human checkpoint going stagnant, with the agent waiting indefinitely for an approval that never comes. Add timeouts. Add an SLA. Ping the human, send them an email about the pending review. Define the timeout behaviour upfront: auto reject, escalate, or whatever fits. Do not leave an agent hanging.

## 6.4 Security

- **Give every agent boundaries.** This is what the session did in practice: each agent could only call a scoped set of tools, and no agent had access to everything.
- **Scope tool permissions per agent** rather than handing the full toolkit to all of them. Principle of least privilege, again.
- **Treat output from a sub-agent as untrusted input** and put a validation layer in front of it. Guardrails and validation via middleware are covered later.
- **Summarise rather than passing all context.** The `Command` plus last-message pattern from the hands-on is exactly this.

## 6.5 Evaluation

Do not just trust everything the agent says. Add an eval layer. Combine LLM as a judge with heuristic evals. Cost evaluation and latency evaluation matter alongside accuracy. This is Session 9.

### Key Insight

A recurring question in the chat was whether you can get good results from cheaper models by decomposing the task. The mentor's answer: that was the entire idea behind hierarchical decomposition. Cheap sub-agents plus a moderate supervisor, together performing a complex task at lower cost. It usually works, but only if the context shared between the nodes is really well defined.

## 7. Session Output and Next Steps

| Item                              | Detail                                                                                                                                                                                                           |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>What you built</b>             | A working supervisor-pattern multi-agent system with four specialised GitHub sub-agents that takes real actions on a live repository.                                                                            |
| <b>Homework, part 1</b>           | Improve the GitHub agent. The mentor named the specific weaknesses himself: the tool grouping is incomplete, the naming is poor (there should be a proper PR agent), and the system prompts can be strengthened. |
| <b>Homework, part 2</b>           | Build something of your own and share it on Discord. Two people will be given the stage next week to present what they built. "Bring me some demos next week."                                                   |
| <b>Try it with a better model</b> | Substitute your own model if you have access to one. A stronger model handles the multi-step documentation plus PR task far more reliably than gpt-oss-120b.                                                     |
| <b>Reading shared</b>             | Google has a good blog on multi-agent architectures, which the mentor offered to post in Discord. A harness engineering video from YouTube was also promised for the validation topic.                           |
| <b>Coming up</b>                  | Session 9 covers evals, guardrails and observability. Context engineering, middlewares, MCP, and the migration of this project off langchain-community all come later.                                           |

On the question of whether this is representative of real industry work, the mentor's view was that the session is very much at par with what is happening in the industry. What is missing is the production hardening layer, evals, guardrails, observability and context engineering, and that is exactly what the remaining sessions cover.

## 8. Quick Revision Summary

| Term                                | What it means                                                                                                                       |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Multi-agent system</b>           | Several specialised agents, each a copy of the single agent architecture, coordinated to solve a task one agent could not.          |
| <b>Test time compute</b>            | Spending compute at inference rather than only at training, so the model can complete difficult, long running tasks.                |
| <b>Context isolation</b>            | Giving each sub-agent only the context it needs, which lowers cognitive load on the model and limits context poisoning.             |
| <b>ReAct loop</b>                   | The reason and act loop running inside every agent. It reasons with the LLM and acts with its tools.                                |
| <b>Agentic workflow</b>             | A workflow where only some nodes are agentic. Overall it is still a graph you designed, but parts of it have agency.                |
| <b>Sequential pattern</b>           | Agents run one after another, each consuming the previous output. Cheapest, most predictable, easiest to debug.                     |
| <b>Coordinator pattern</b>          | Also supervisor or orchestrator. A boss agent dynamically delegates to sub-agents that never talk to each other.                    |
| <b>Hierarchical pattern</b>         | Coordinators of coordinators. Most expensive, least predictable, needs the most context engineering.                                |
| <b>Chinese whispers problem</b>     | Why hierarchical systems are unpredictable. A poisoned message lives in the manager's history and trickles into every sub-agent.    |
| <b>Agent as a tool</b>              | The supervisor pattern in code: wrap a sub-agent in a function, decorate it with @tool, and pass it to the supervisor's tools list. |
| <b>Command</b>                      | The LangChain and LangGraph mechanism for injecting a message into a pipeline, used here for agent to agent communication.          |
| <b>Principle of least privilege</b> | Give each agent only the tools it needs, exactly as you would scope a user's permissions.                                           |
| <b>Tool renaming</b>                | Converting human readable tool names into function-like, AI native names so the model can call them reliably.                       |
| <b>CRAFT</b>                        | Context, Role, Action, Format, Tone. The structure for writing a sub-agent system prompt.                                           |
| <b>Idempotency</b>                  | Running a node twice produces the same result, like CREATE TABLE IF NOT EXISTS. A design goal for graph nodes.                      |

| Term              | What it means                                                                                                                  |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>GitHub App</b> | The authentication and permission layer between your agent and a specific repository.                                          |
| <b>uv</b>         | A Python package manager, comparable to npm, Maven or Gradle. <code>uv sync</code> installs dependencies and creates the venv. |